Welcome, Shane! I'm Claude.

Bring me anything—a tough problem, a half-formed idea, something you need to write.
We'll figure it out together.

Where do you want to start?

Can you build me a texas holdem poker website written in python using the Drafter UI that follows my instructions: I want it to contain four state dataclass attributes:
players: List[Player]. This represents the user and two computer opponents. Modification: Populated at game start, modified during betting rounds to update chip counts, current bets, and fold statuses.
Community_cards: List[str]. This represents the five cards placed in the middle Modification: Cleared at the start of each hand; appends 3 cards on the flop, 1 card on the turn, and 1 card on the river.
Pot: int. This represents total bets Modification: Increases as players call, raise, or post blinds; resets to 0 when awarded to the winning player at showdown.
Current_turn_index: int. An integer index representing whose turn it is Modification: Increments after every action; wraps back to 0 using modulo logic until the betting round closes.

I also want it to contain a second dataclass with these elements: name (str): The display name of the player (e.g., "User", "Bot 1", "Bot 2").

- Modification: Set once during initial state setup; remains read-only during gameplay.

chips (int): The total remaining stack of chips available to bet.

- Modification: Decreased when placing bets/blinds; increased when winning a hand or reset via game restart.

hole_cards (List[str]): The private 2-card hand dealt to the player.

- Modification: Cleared and reassigned 2 new card strings at the start of every hand.

current_bet (int): The amount of chips placed into the active betting round by this player.

- Modification: Increases during fold/call/raise actions; resets to 0 when transitioning to the next street (e.g., Flop to Turn).

is_folded (bool): Flag indicating whether the player has surrendered their hand.

- Modification: Defaults to False; flips to True when the player selects "Fold" or when an AI opponent folds. It also needs 7 route functions being: index — Renders the main game table and controls.

- start_new_game — Resets player stacks, clears community cards, and sets up a fresh deck.
- deal_hand — Deals 2 hole cards to each player and posts initial small/big blinds.
- player_action — Handles human actions (fold, call, raise) submitted via the user interface.
- trigger_bot_turns — Executes decision-making logic for AI opponents when current_turn_index belongs to a bot.
- advance_street — Transitions the game phase (Pre-flop → Flop → Turn → River → Showdown) and deals community cards.
- reset_stack — Recharges the human user's chip stack if they run out of money. include these page functions: Welcome / Setup Page: Allows the player to enter their name, select a starting chip stack size, and choose the difficulty level of the two AI opponents.
- Main Poker Table Page: The primary screen showing community cards, the pot, the user's hand, AI card backs, player chip counts, and action buttons.
- Showdown / Hand Results Page: Highlights the winning player, displays all opponent hole cards, reveals winning 5-card hand rankings, and awards the pot.
- Game Over / Rebuy Page: Displayed when the user runs out of chips, featuring statistics for the session and a button to re-buy or start over.
- Rules & Hand Rankings Guide: A reference page explaining Texas Hold'em hand hierarchies (High Card to Royal Flush) and basic game rules. I want these for user inputs: Action Choice (SelectBox):
  - Purpose: Allows the player to select their main action during their turn (Check/Call, Raise, Fold). Raise Amount (TextBox):
  - Purpose: Accepts a numeric entry for the custom chip amount the user wants to raise when Raise is selected. Auto-Muck Losing Hands (CheckBox):
  - Purpose: A toggle allowing the user to automatically hide/muck their losing hand at showdown rather than revealing it to opponents. I also want it to contain these three if statements: Validating Player Raise Amount:
    - Logic Evaluated: Checks if the entered raise amount is both greater than the current minimum raise and less than or equal to the player's available chips.
    - Site Impact: If valid, deducts chips and advances the turn; if invalid, prevents submission and displays an error message on the table page. Determining Hand Advancement vs. Showdown:
    - Logic Evaluated: Checks if all non-folded players have matched the highest current_bet in the active round.
    - Site Impact: If true and less than 5 community cards are present, invokes advance_street. If 5 community cards are out, routes the site to the Showdown Page. Evaluating Uncontested Winner (Fold Detection):

- Logic Evaluated: Evaluates whether the number of active players with is_folded == False equals 1.
- Site Impact: Skips remaining community card deals immediately, awards the pot to the last remaining player, and redirects to the Hand Results Page. It also needs a loop: Description & Purpose: This loop iterates through the list of Player instances stored in state.players to evaluate if every active player has matched the current highest bet or gone all-in.
- Why It Is Meaningful: It governs core Texas Hold'em game flow. Without iterating through state.players, the app cannot determine when a betting round is legally finished, preventing unauthorized state progression to the Flop, Turn, River, or Showdown.

Show less

Working out how side pots get awarded ›

I'll build this as a single Python file using Drafter, then test the game logic. First I'm checking whether Drafter is installed here.

Updated tasks, ran a command ›

Drafter isn't installed here and package installs are blocked, so I can't run the real UI. I'll write the site against Drafter's API and test the game engine with a stand-in `drafter` module.

Created 2 files, edited a file, and 9 more steps ›

I've built the poker site in `poker_site.py`, but I haven't seen it run in a real Drafter window. Drafter isn't installed in my environment and package installs are blocked. I tested the game logic with a stand-in `drafter` module instead. Run it with `pip install drafter` and then `python poker_site.py`.

What I tested. I played about 1,500 simulated hands at all three difficulties. I checked that chips are never created or lost, the pot resets to 0 after every hand, and hands never get stuck. I also checked the hand ranker on every category (including A-2-3-4-5 straights and 7-card best-hand selection), side pots, split pots, and the fold, raise and auto-muck rules.

What's in it:

- **Dataclasses:** `Player` has your five fields exactly. `State` has your four required fields plus supporting ones the game needs (the deck, the dealer seat, blinds, results of the last hand and session stats).
- **Routes:** all seven you listed.
- **Pages:** welcome/setup, table, showdown results, game over/rebuy, and rules with a hand-ranking table.
- **Inputs:** `SelectBox` for the action, `TextBox` for the raise amount, `CheckBox` for auto-muck.

- **The three `if` statements:** they sit in `player_action` (raise validation) and `check_progress` (fold detection first, then advance-vs-showdown).
- **The loop:** `betting_round_complete` iterates `state.players` to decide whether the round is over.

Choices I made:

- **Extra route:** I added an eighth route, `show_rules`, so the Rules page is reachable from a button. Drop it if your assignment needs exactly seven.
- **Raise rule:** a raise must be *at least* the minimum raise, not strictly greater. The amount is chips added on top of the call, and the table page shows the legal range.
- **Opponents:** the bots estimate their winning chances by simulation. Easy bots are noisy and loose, Hard bots are accurate and bluff sometimes. Busted bots re-buy automatically; you get the Game Over page instead.
- **Blinds:** big blind is 1/50 of the starting stack, small blind is half that.

Drafter version: I wrote the Drafter calls from memory. It uses `Header`, `Table`, `Button`, `TextBox`, `SelectBox`, `CheckBox`, `LineBreak` and `HorizontalRule`. If your version names any of these differently, it will fail when that page renders, so check that first.

can you keep this original design it is great, but add a button that lets you return back to the start screen. also color code suits making diamonds orange, hearts red, clubs green, and spades dark blue

Rewriting card-printing helpers for colored suits ›

I'll add a "Quit to Start Screen" button on every game page and color the suits (diamonds orange, hearts red, clubs green, spades dark blue), leaving everything else as is.

Edited a file, shared a file, ran 2 commands ›

I've updated `poker_site.py` with a button back to the start screen and color-coded suits. The game design and rules are unchanged. I tested both with a stand-in for Drafter, not the real one, so check the colors in your browser first.

Return to the start screen. A new route, `return_to_start`, sends you back to the welcome/setup page. There is a "Quit to Start Screen" button on the table and results pages, and a "Back to Start Screen" button on the Game Over and Rules pages. Quitting abandons the current game, since "Start Game" always begins a fresh one. I can add a "Resume Game" option on the start screen if you'd rather not lose a game by accident.

Suit colors. Every card is colored by suit wherever it appears: the community cards, your hand, the bots' revealed cards at showdown, the winning five-card hand, and the examples on the Rules page. Diamonds are orange, hearts red, clubs green and spades dark blue.

Colors in your Drafter. The colors use Drafter's `Span` and `change_color`, which I wrote from memory. If your version doesn't have them, the cards show in plain black text instead of crashing, so a missing color means a version mismatch. If a color looks off, such as the dark blue reading as black, change the hex value in `SUIT_COLORS` near the top of the file.

can you keep it the exact same, but make it 5 players, and make the AI opponents smarter?

Finishing the call and fold thresholds for the postflop bot logic ›

I'll make the table 5 players and rebuild the bot brain. I'll keep the UI, routes, dataclasses and rules as they are. Bots will use a position-aware pre-flop chart, simulated win chances against the opponents' likely hands, and a running read on how often each player raises.

Edited a file, ran 2 commands, updated tasks ›

there are still only two bots. There should be one user, and four bots with rotationg jobs representing small blind, big blind, and dealer. additionally improve the AI by having the bots displayed around a table with cards shown

there are still only two bots. There should be one user, and four bots with rotationg jobs representing small blind, big blind, and dealer. Additionally, edit the UI by placing the four players around a table, and the cards players should have should display as an image of the card. when you submit me the updated python file, also give me all image files used to update the UI

Check Pillow, fonts and current player count ›

Pillow and DejaVu fonts are available, so I'll draw all 52 card faces, a card back and the dealer/blind badges. They'll be saved as PNG files in an `images` folder next to the site.

Created a file, read a file, ran 2 commands ›



The new card art is clean, with no collisions. Now I'll rebuild the table page. First I'm reading the current page code so my edits match it.

Shared 3 files, read 2 files, ran 5 commands ›



You were seeing only two bots because I never sent you the 5-player file until now. The table now has you plus four bots, with card images and the layout you asked for. I tested it against a stand-in for Drafter, so it hasn't run in real Drafter yet.

Setup. Unzip `poker_images.zip` into the same folder as `poker_site.py` so you get an `images` folder beside it. Then run `python poker_site.py`. If you delete the images, it falls back to plain colored text instead of crashing.

What changed:

- **Players:** you plus Bot 1 to Bot 4.
- **Rotating jobs:** the dealer button, small blind and big blind move one seat clockwise every hand. Each seat shows a round D, SB or BB badge and the role name. In my test the three roles moved around the table as expected over six hands.
- **Table layout:** Bot 2 and Bot 3 sit across the top, Bot 1 and Bot 4 on the sides, and you at the bottom. The pot and the five community card slots sit in the middle. The seat whose turn it is turns yellow, folded seats turn gray, and the winner turns gold.
- **Card images:** your cards, the community cards, the bots' cards (face-down card backs) and the winning five-card hand on the results page are all pictures. Bots' cards flip face-up at showdown. The results page shows the table again with the winning hand and a results table.

Image files. `poker_images.zip` holds all 57 images:

- 52 card faces, named like `AS.png` (ace of spades) and `TH.png` (ten of hearts), in the suit colors you chose
- `back.png`, the card back
- `empty.png`, the slot for community cards not yet dealt
- 3 role badges
- `make_card_images.py` is included so you can redraw or restyle them.

What I couldn't confirm in real Drafter. The layout uses Drafter's `Image`, `Div` and styling calls, which I wrote from memory like the earlier ones. Two things could differ in your version: whether local image paths like `images/cards/AS.png` load, and whether a table

cell can hold a block of images and text. If the cards don't appear, let me know and send me any error.

I also made the bots smarter, which didn't make it into my last message. They now use a pre-flop hand chart based on position and track how often each player raises. They bluff, slow-play strong hands, and estimate hand strength against what raisers likely hold. In bot-versus-bot games Hard beat Easy by a clear margin, though Medium and Hard are close to each other. In a table of Hard, Medium, Easy and two simple baseline bots, Easy ended up ahead of Hard, so the three difficulty levels aren't cleanly ordered. Ask if you want that tuned.

Download all

When I ran the Poker site.py file the green background looked good but I was unable to actually view the images of cards. It just displayed the no image icon

Checking how Drafter encodes local images as data ›

That "no image" icon means the browser couldn't load the files, so my image paths don't match how Drafter finds local images. I'll check how Drafter handles them instead of guessing again.

Read 3 pages, searched the web, 3 failed ›